

Parallel & Distributed Computing



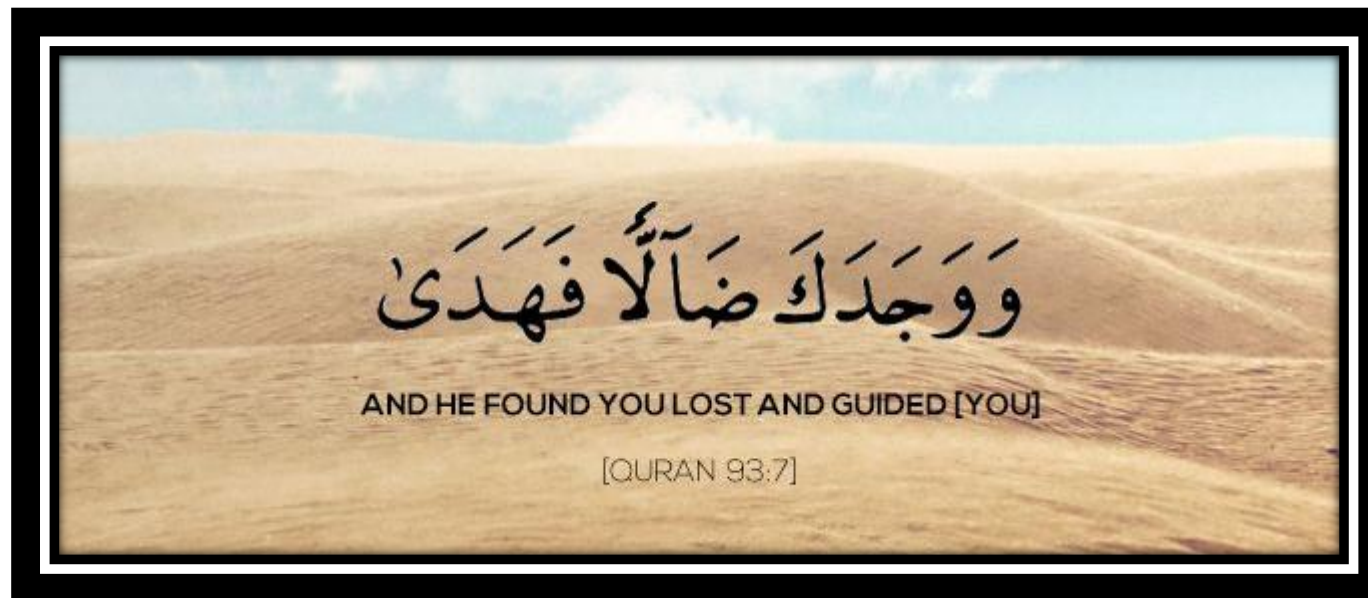
PARALLEL ALGORITHMS DIVIDE AND CONQUER



Dr. Muzamil Ahmed (Assistant Professor)
Namal University Mianwali

Lesson from Quran

2



Lecture Outline

3

- Parallel Algorithms
- Sequential & Concurrent Algorithms
- Divide and Conquer
- Sequential vs Parallel Divide and Conquer
- Divide and Conquer Recurrence – Theory
- Sequential vs Parallel Time
- Matrix Multiplication – Strong Parallel Case
- Common Mistakes in Parallel Divide and Conquer

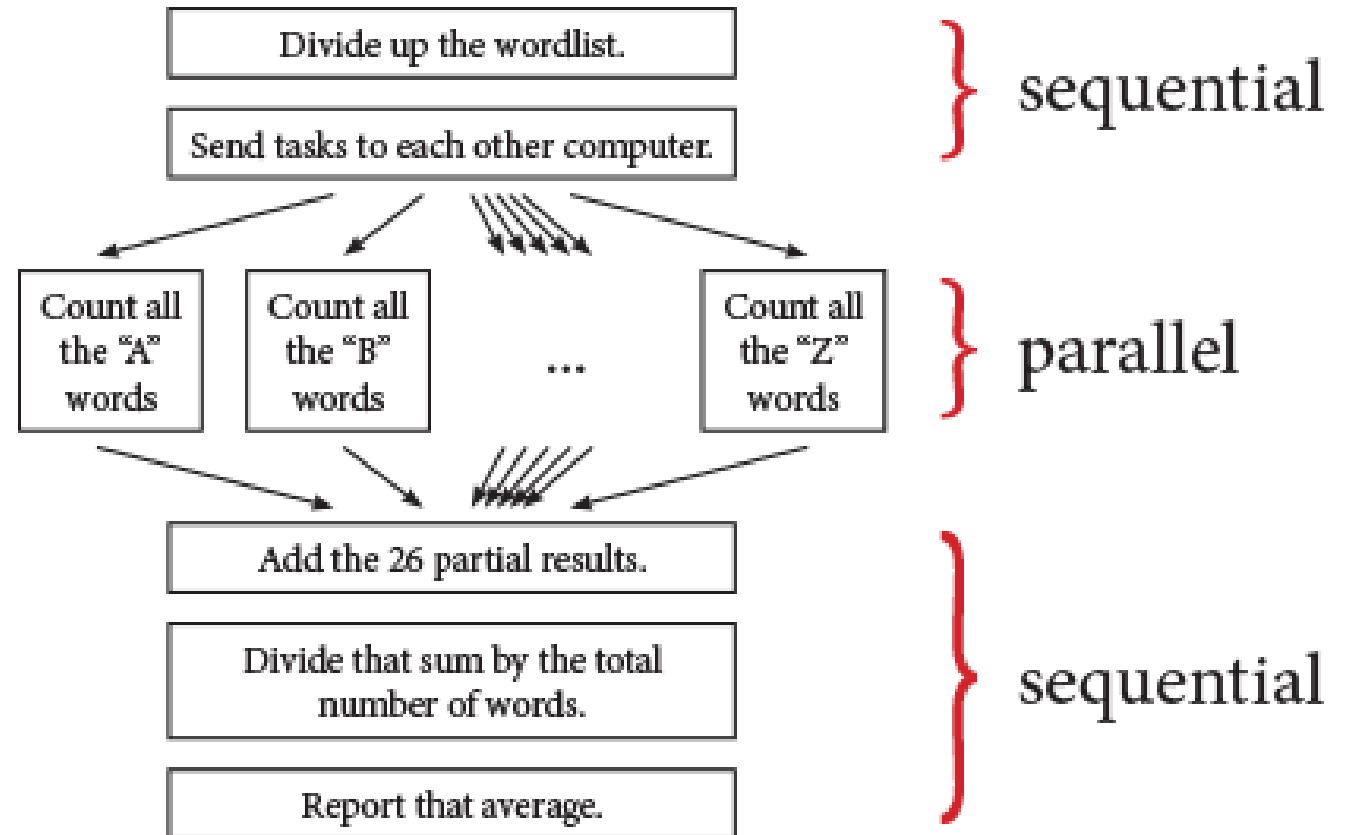


What Are Parallel Algorithms?

- A **parallel algorithm** is an algorithm designed to execute multiple operations simultaneously using multiple processing elements.
- **Key Characteristics:**
 - ▣ Exploits concurrency in the problem (
 - ▣ Uses multiple cores/processors
 - ▣ Reduces execution time compared to sequential algorithms
- If:
 - ▣ Sequential time = $T_1(n)$
 - ▣ Parallel time on p processors = $T_p(n)$
- Then:
 - ▣ **Speedup:** $S_p = T_1(n) / T_p(n)$
 - ▣ **Efficiency:** $E_p = S_p / p$ (Values range from **0 to 1**)

Sequential Algorithms

- An algorithm in which some consecutive steps of instructions are executed in a chronological order to solve a problem.



Parallel Algorithms

- A **parallel algorithm** is a step-by-step problem-solving procedure that:
 - ▣ Identifies independent computations
 - ▣ Schedules them to run simultaneously
 - ▣ Minimizes coordination and communication
- **Key Design Goals:**
 - ▣ Correctness
 - ▣ Performance improvement
 - ▣ Scalability
- **Core Idea:** Hardware does not create parallelism — algorithms do.

Why Divide and Conquer is Important in Parallel Computing

- Divide and Conquer (D&C) is one of the **most parallel-friendly algorithmic paradigms**.
- **Three Core Steps:**
 - ▣ **Divide** the problem into smaller subproblems
 - ▣ **Conquer** subproblems recursively
 - ▣ **Combine** the solutions
- **Parallel Insight:**
 - ▣ Subproblems are often independent
 - ▣ Independent tasks can be executed concurrently
- **Examples:** Merge Sort, Quick Sort, Binary Search, Matrix Multiplication

Parallelism?

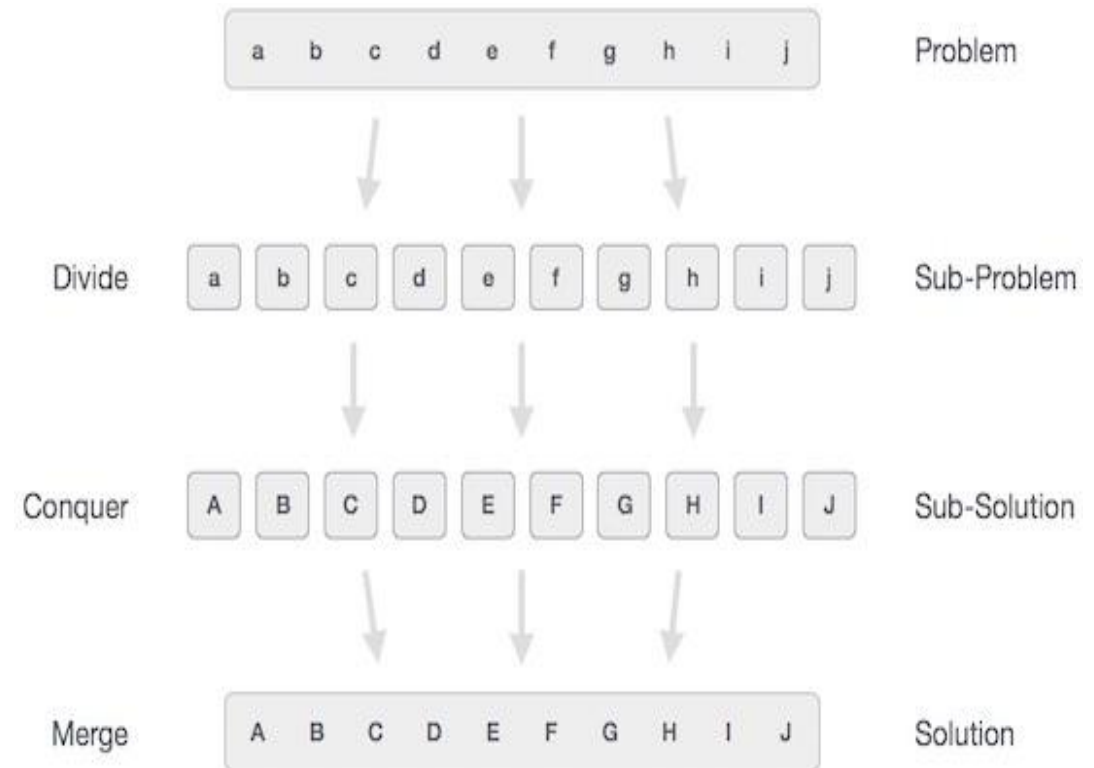
8

- Parallelism means performing **multiple useful computations at the same time** to solve a problem faster.
- Parallelism is NOT about using threads or OpenMP first — it is about **how a problem is structured**.
- **Types of Parallelism:**
 - ▣ *Data Parallelism*: same operation on different data items
 - ▣ *Task Parallelism*: different tasks executed concurrently
 - ▣ Divide and Conquer primarily exposes **task parallelism**.

Why Divide & Conquer Is a Natural Fit for Parallelism

9

- **Divide and Conquer follows a recursive mindset:**
 - ▣ Break a big problem into smaller pieces
 - ▣ Solve each piece independently
 - ▣ Combine results
- **Why this matters for parallel systems:**
 - ▣ Independence = concurrency
 - ▣ Recursion = repeated parallel structure
 - ▣ Smaller problems = better cache usage
- This makes Divide and Conquer one of the **most important paradigms** in parallel algorithm design.



Sequential Divide and Conquer – Conceptual Flow

10

- In a **sequential environment**, divide and conquer still works, but:
- Subproblems are solved one after another
- CPU remains idle during recursive waits
- **Example:** Merge Sort on a single core
 - ▣ Left half sorted first
 - ▣ Right half sorted later
 - ▣ No true overlap of computation occurs.

Parallel Divide and Conquer – Conceptual Shift

11

- In a **parallel environment**, the mindset changes:
 - ▣ Subproblems do NOT wait for each other
 - ▣ Recursive calls can execute simultaneously
 - ▣ Processor cores stay busy
- **Key Insight:**
 - ▣ If two tasks do not depend on each other, they *should not* be executed sequentially.

Work and Span (Intuition)

12

- To analyze parallel algorithms, we separate two concerns:
 - ▣ **Total Work (T_1):** How much computation exists overall?
 - ▣ **Critical Path / Span (T_∞):** What is the longest chain of dependencies?
- Even with infinite processors, execution time cannot be less than the span.

Example: Sorting a Large Dataset

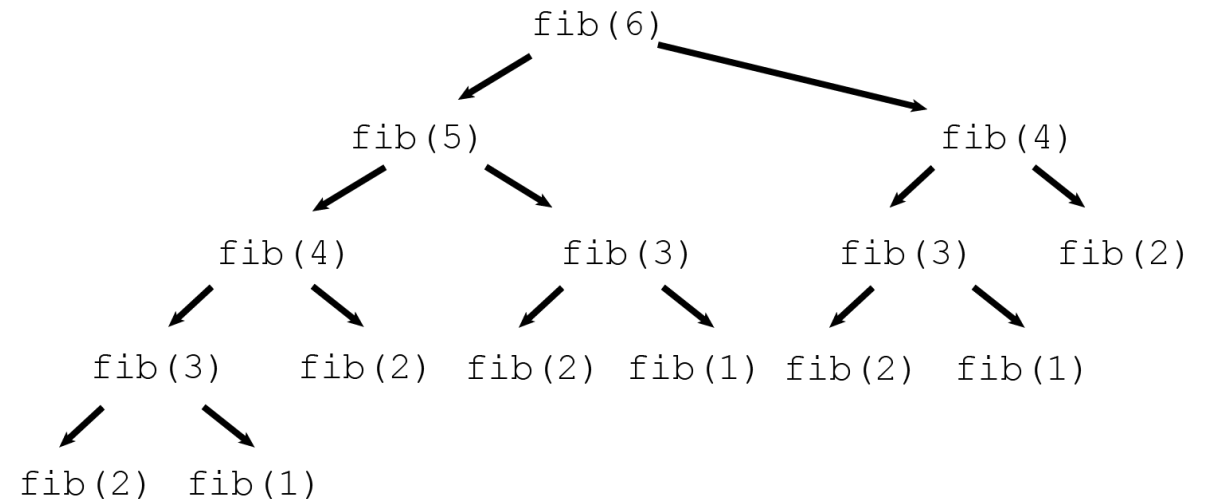
13

- Suppose we need to sort 1 million numbers.
- **Sequential Thinking:**
 - ▣ One CPU sorts everything step by step
- **Parallel Divide & Conquer Thinking:**
 - ▣ Split data into chunks
 - ▣ Different cores sort different chunks
 - ▣ Results are merged
 - ▣ This idea applies across many domains: graphics, AI, databases.

Example: Recursion Trees

14

- Divide and conquer algorithms are best understood using **recursion trees**.
- Each node represents:
 - ▣ A subproblem
 - ▣ A computation task
- **Parallel View:**
 - ▣ Nodes at the same level can run simultaneously
 - ▣ Tree height limits parallel speed



Divide and Conquer Recurrence – Theory

15

- General recurrence relation:

$$T(n) = aT(n/b) + f(n)$$

- **Meaning:**

- ▣ a subproblems
 - ▣ Each of size n/b
 - ▣ $f(n)$ cost to divide and combine
- This formula captures the *structure* of the algorithm, not hardware details.
- This recurrence captures only the algorithm's structure — not hardware details.

Sequential vs Parallel Time – Conceptual Difference

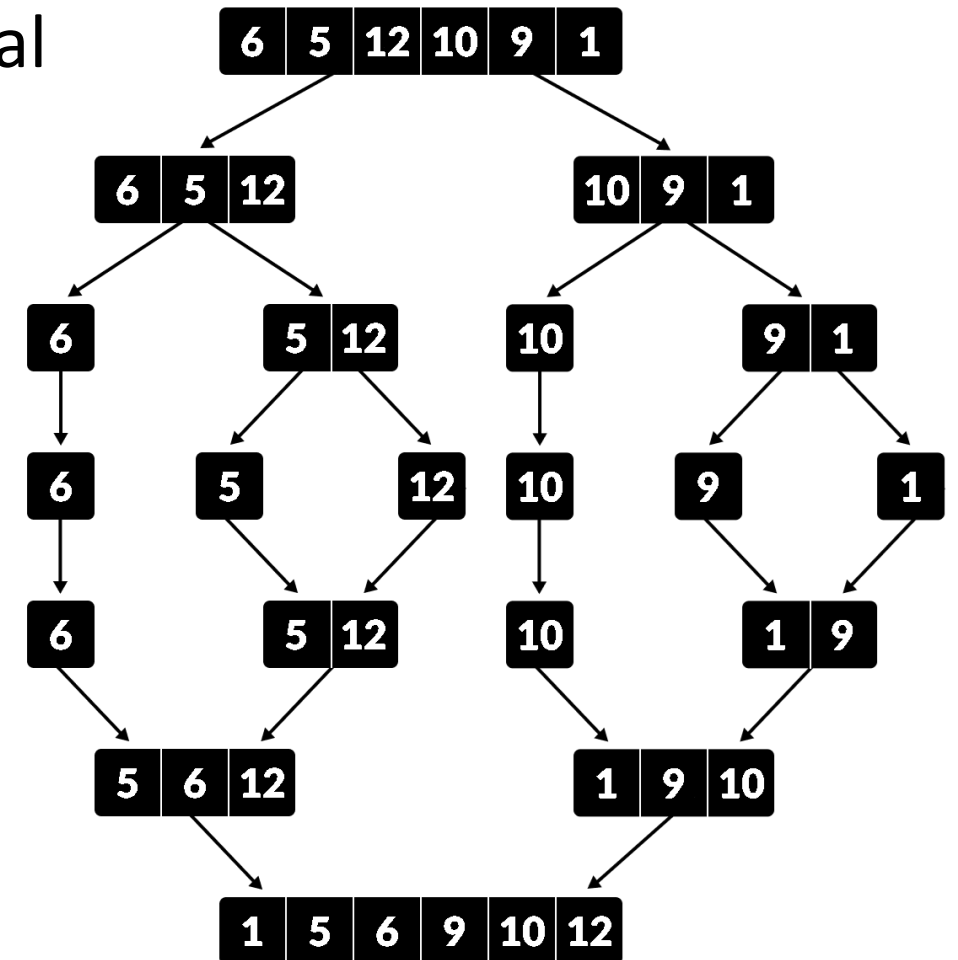
16

- **Sequential:** All recursive calls add up
- **Parallel:** Only the *slowest* recursive chain matters
- **Result:** Parallel time is dominated by:
 - ▣ Recursion depth
 - ▣ Combine step cost
- This is why balancing matters.

Example: Merge Sort

17

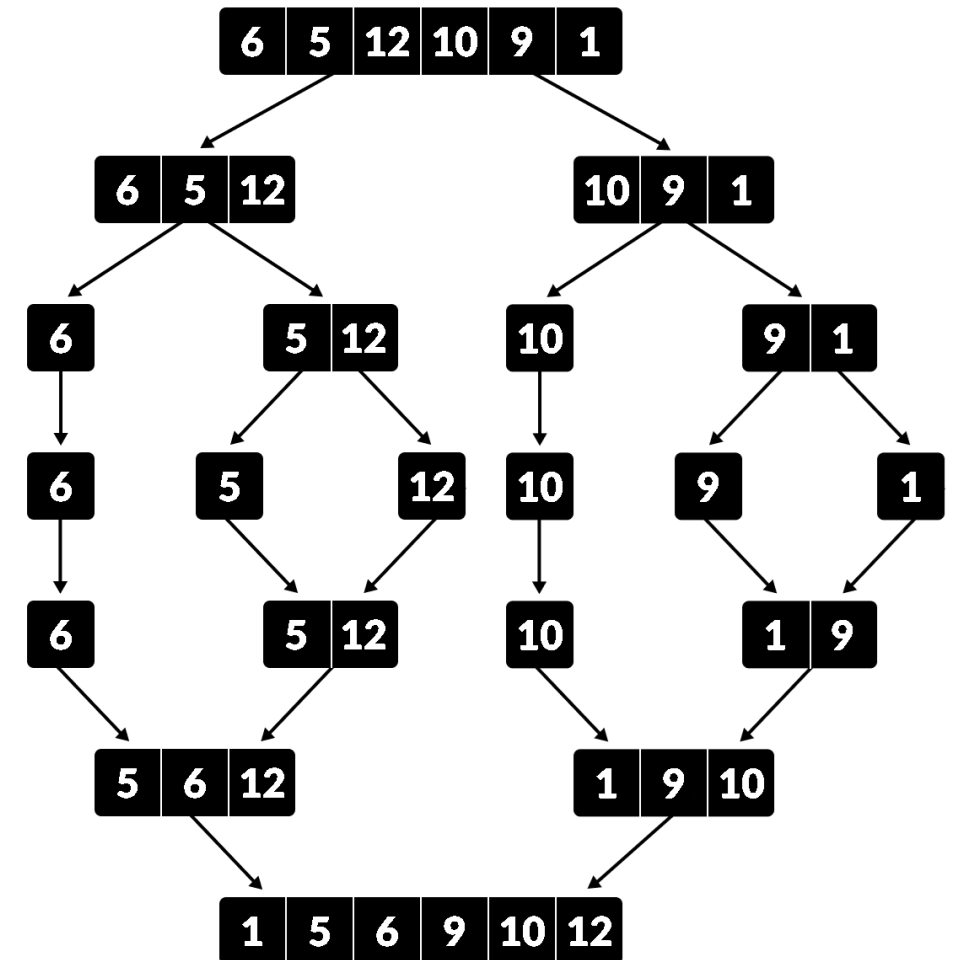
- Merge Sort divides an array into two equal halves repeatedly.
- **Why it is good for parallelism:**
 - ▣ Balanced recursion tree
 - ▣ Equal-sized subproblems
 - ▣ Predictable structure
- **Why it is challenging:**
 - ▣ Merge step requires coordination
- Good parallel algorithm $\neq$ perfect parallel algorithm.



Parallel Merge Sort – High-Level Pseudocode

18

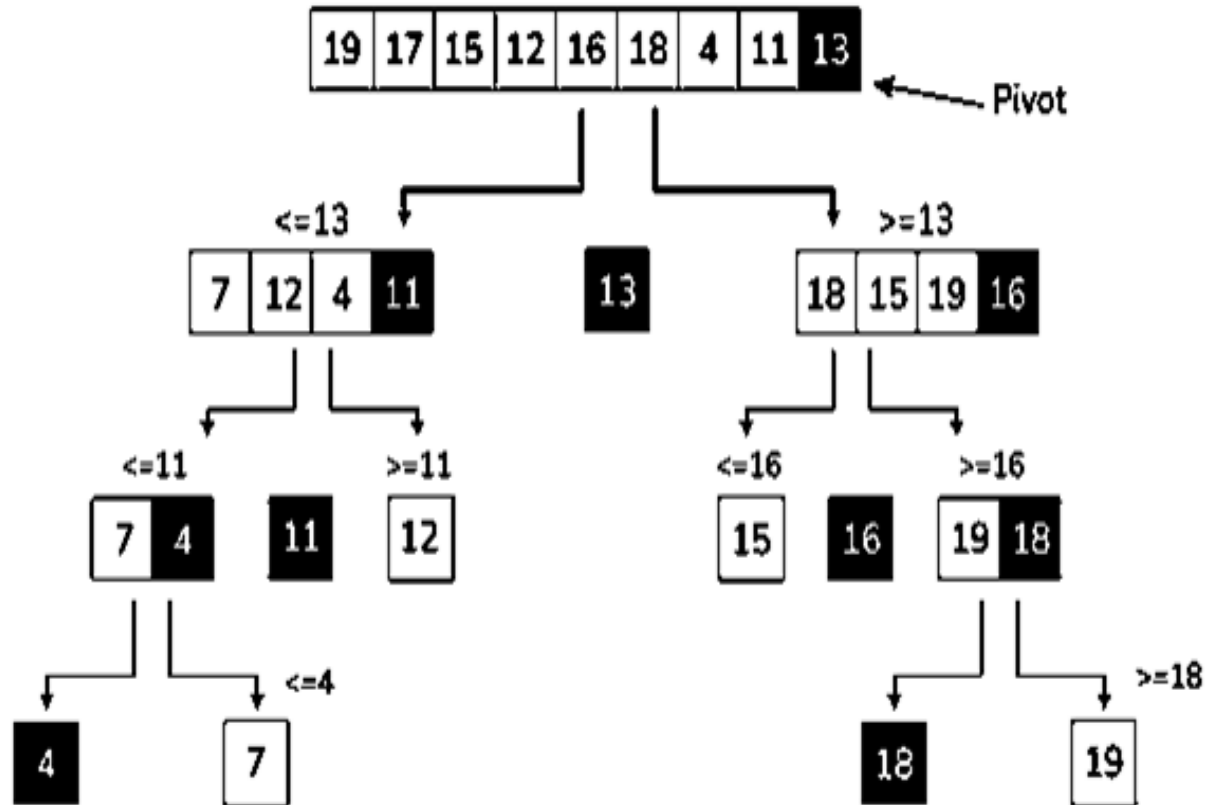
```
function parallel_merge_sort(A) :  
    if A is small:  
        sort sequentially  
    else:  
        split A into A1 and A2  
        run parallel:  
            parallel_merge_sort(A1)  
            parallel_merge_sort(A2)  
        merge A1 and A2
```



Quick Sort – Why Parallelism Is Harder

19

- ❑ Quick Sort also uses divide and conquer, but:
 - ▣ Partition step is sequential
 - ▣ Subproblem sizes may be uneven
- ❑ Not every divide-and-conquer algorithm scales equally well.
- ❑ Algorithm structure matters more than recursion alone.



Quick Sort – Algorithm and Parallel Impact

20

□ Sequential Quick Sort:

QuickSort(A):

if size(A) $\leq$ 1:

return A

pivot = choosePivot(A)

partition A around pivot

QuickSort(left)

QuickSort(right)

□ Parallel Version:

ParallelQuickSort(A):

if size(A) $\leq$ threshold:

sort sequentially

else:

partition A

spawn ParallelQuickSort(left)

spawn ParallelQuickSort(right)

sync

□ Parallel Effect:

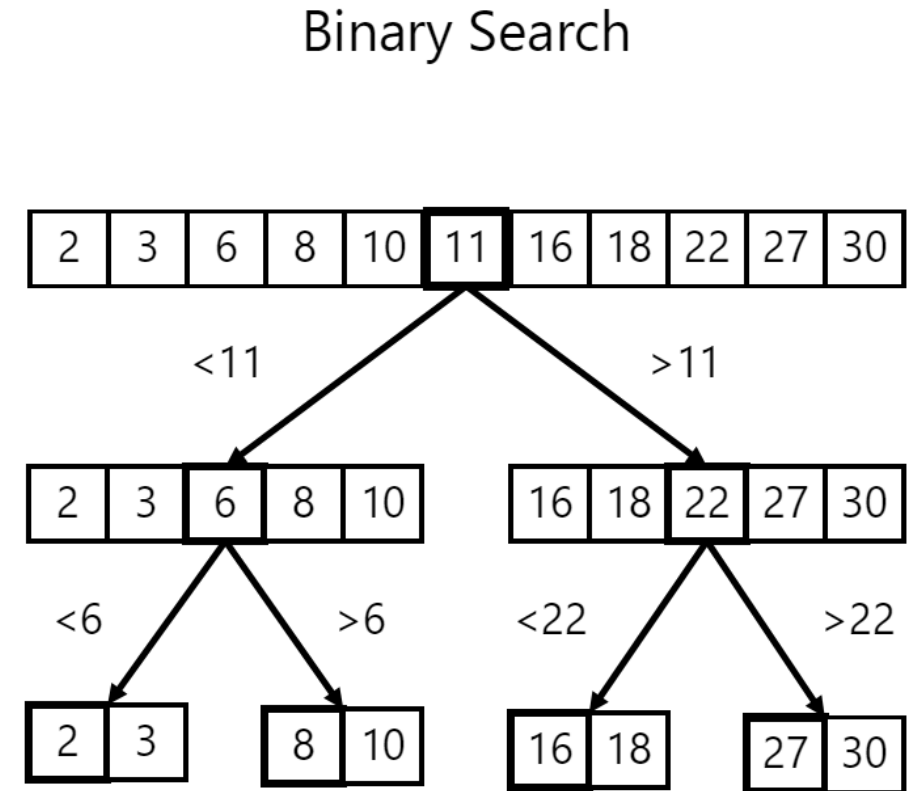
Recursive calls parallel

Partition step remains sequential → limits speedup

Binary Search – Limited Parallelism

21

- Binary search divides the problem, but:
 - ▣ Only one subproblem survives
 - ▣ No independent work remains
- Divide and conquer does NOT automatically mean parallelizable.



Binary Search – Limited Parallelism

22

□ **Parallel Analysis:**

- ▣ Only one recursive path survives
- ▣ No independent subproblems
- ▣ Divide and conquer $\neq$ parallelizable

BinarySearch(A, x):

 mid = size(A)/2

 if A[mid] == x:

 return mid

 else if x < A[mid]:

 search left half

 else:

 search right half

Matrix Multiplication – Strong Parallel Case

23

- Matrix multiplication divides data into blocks.
- **Why it scales well:**
 - ▣ Many independent computations
 - ▣ High computation-to-communication ratio
- **Conceptual Takeaway:** Best parallel divide-and-conquer algorithms have minimal dependencies.

		b_{11}	b_{12}	b_{13}
		b_{21}	b_{22}	b_{23}
		Matrix B		
	a_{11}	a_{12}	$a_{11} \times b_{11} + a_{12} \times b_{21}$	$a_{11} \times b_{12} + a_{12} \times b_{22}$
	a_{21}	a_{22}	$a_{21} \times b_{11} + a_{22} \times b_{21}$	$a_{21} \times b_{12} + a_{22} \times b_{22}$
	a_{31}	a_{32}	$a_{31} \times b_{11} + a_{32} \times b_{21}$	$a_{31} \times b_{12} + a_{32} \times b_{22}$
	Matrix A		Matrix C	


$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 10 & 11 \\ 20 & 21 \\ 30 & 31 \end{bmatrix}$$
$$= \begin{bmatrix} 1 \times 10 + 2 \times 20 + 3 \times 30 & 1 \times 11 + 2 \times 21 + 3 \times 31 \\ 4 \times 10 + 5 \times 20 + 6 \times 30 & 4 \times 11 + 5 \times 21 + 6 \times 31 \end{bmatrix}$$
$$= \begin{bmatrix} 10 + 40 + 90 & 11 + 42 + 93 \\ 40 + 100 + 180 & 44 + 105 + 186 \end{bmatrix} = \begin{bmatrix} 140 & 146 \\ 320 & 335 \end{bmatrix}$$

Common Mistakes in Parallel Divide and Conquer

24

- ❑ Assuming recursion = parallelism
- ❑ Ignoring combine-step cost
- ❑ Creating excessive threads
- ❑ Forgetting load imbalance
- ❑ Parallel thinking requires discipline.

When to Use Parallel Divide and Conquer

25

□ **Use when:**

- ▣ Subproblems are independent
- ▣ Work is evenly distributed
- ▣ Problem size is large

□ **Avoid when:**

- ▣ Dependencies dominate
- ▣ Overhead exceeds computation

□ **Key Takeaways:**

- ▣ Divide and conquer exposes task parallelism
- ▣ Theory guides design decisions
- ▣ Code is the *last* step, not the first

Thanks

26

“

The only way to do great
work is to love what
you do.

Steve Jobs